Testing Artifact - White Box Testing

SENG401 - Final Project

Victor Gouttin

2026-03-27

Overview

This artifact documents the white box testing (code-based testing) for the logic of our game "Chronicles of Change". This game was built using an engine called Ren'Py, which is primarily used for visual novel games. In our case, we adapted it to have hands-on gameplay as well as a storyline with information that results in quizzes after each level.

A key challenge in this testing was that I wanted to only test the logic we had created, and not the Ren'Py methods, which presumably already work. In order to achieve this I had to overcome the fact that .rpy files cannot be imported into regular .py files since there are unique Ren'Py elements that don't transfer over. To work around this, I copied the logic from every source code file and made an equivalent .py file inside the tests directory. The full directory for the tests in the GitHub is: rpgfight/game/tests. Here there are 4 test files which I will describe shortly.

test_modifier_system.py

Test Case	Description	Expected Result	Actual Result
test_add_modifier	Add one modifier to ModifierSystem, check active_modifiers length	len == 1	PASS
test_get_total_single	Add one HEALTH_BOOST of 50, check get_total()	Total == 50	PASS
test_get_total_multiple	Add two HEALTH_BOOSTs of 50, check get_total()	Total == 100	PASS
test_get_modifier_names	Add HP+ and Speed+ modifiers, check names list	Both names in list	PASS
test_clear_modifiers	Add two modifiers then clear(), check active_modifiers	len == 0	PASS
test_apply_to_player	Apply HEALTH_BOOST of 50 to MockPlayer with 100 base HP	health == 150, max_health == 150	PASS

modifier_system.rpy defines the game's modifier system, meaning the extra stats that you can gain or lose depending on if you do well in the quiz or not. There are two classes, Modifier which has the different types of stat changes, and ModifierSytstem which manages the active modifiers.

test_player.py

Test Case	Description	Expected Result	Actual Result
test_player_initialization	Create player, check health and velocity	health == STARTING_HEALTH, velocity (0,0)	PASS
test_player_jump_sets_velocity	Player stands on MockPlatform and jumps	velocity.y < 0, animate_jump == True	PASS
test_player_move_right	Set keyboard.right = True, call move()	acceleration.x > 0	PASS
test_player_move_left	Set keyboard.left = True, call move()	acceleration.x < 0	PASS
test_player_fire_reduces_dash	Player fires with 1 dash remaining	dashes_remaining == 0	PASS
test_player_refill_dashes	Set dashes_remaining = 0, call refill_dashes()	dashes_remaining == dash_count	PASS
test_player_flash	Call start_flash(), simulate flash_duration frames	is_flashing True then False	PASS
test_player_reset	Move player to 999,999 then reset()	position == starting position	PASS

player.rpy contains the Player class which handles all the player behavior including moving, jumping, shooting and taking damage. It manages the physics of the character with vectors.

test_enemies.py

Test Case	Description	Expected Result	Actual Result
test_enemy_initial_state	Create enemy, check is_dead, hp, counters	is_dead False, hp 1, counters 0	PASS
test_enemy_frames_dead_counting	Set is_dead True, run 60 frames of check_animations()	seconds_dead == 1	PASS
test_enemy_no_revive_when_zero	Set revive_seconds = 0, run 300 frames	animate_rise False, is_dead True	PASS
test_enemy_revive_triggers	Set revive_seconds = 3, run 180 frames	animate_rise == True	PASS
test_enemy_revive_resets_stats	Set animate_rise True, call check_animations()	is_dead False, hp 1, counters 0	PASS
test_enemy_collision_repositions	Place enemy over MockPlatform, call check_collisions()	position.y == platform.y - height + 5	PASS

enemies.rpy contains the Enemy class which controls enemy movement, spawning, death and reviving. Enemies walk around the game map and can deal damage to the player, and can revive depending on their configurations.

test_core.py

Test Case	Description	Expected Result	Actual Result
test_vector_add_vector	Add Vector(1,2) + Vector(3,4)	Result (4, 6)	PASS
test_vector_add_scalar	Add Vector(1,2) + 3	Result (4, 5)	PASS
test_vector_mul_vector	Multiply Vector(2,3) * Vector(4,5)	Result (8, 15)	PASS
test_vector_mul_scalar	Multiply Vector(2,3) * 4	Result (8, 12)	PASS
test_vector_rmul_scalar	Multiply 4 * Vector(2,3)	Result (8, 12)	PASS
test_gamesprite_is_colliding_true	Two overlapping GameSprites	is_colliding() == True	PASS
test_gamesprite_is_colliding_false	Two non-overlapping GameSprites	is_colliding() == False	PASS
test_gamesprite_is_colliding_edge	Two sprites touching at exact edge	is_colliding() == True	PASS
test_maptile_collision_padding	Sprite inside padding zone of MapTile	is_colliding() == False	PASS
test_maptile_collision_inside	Sprite well inside MapTile bounds	is_colliding() == True	PASS

core.rpy contains the foundational classes for the entire system. It has the Vector class, GameSprite class and MapTile class which all deal with the physics, collisions, and logic that the game relies on.

***These tables were created by claude.ai, I gave it my source code and it created these tables in a docx file which I took screenshots of and imported them here.**

The unit tests validate that the core game logic behaves correctly. The modifier system tests confirm that stat boosts accumulate and apply to the player as intended, which is critical to the game's progression system. The player tests verify that physics, combat, and state transitions function correctly in isolation. The enemy tests confirm that the revive system correctly tracks time and resets state. The core tests validate the foundational Vector math and collision detection that have to do with all the movement and interaction in the game.

One limitation of this testing approach is that Ren'Py-specific behaviour such as rendering, sound, and input handling cannot be tested with pytest. These components were excluded from testing under the assumption that Ren'Py engine code is reliable, and only the game-specific logic written by the team was subject to validation.